

RAG BUILDING BLOCKS REFERENCE

AI Engineer Ready · Week 7 — The rag/ Folder, End to End

GOLDEN RULE loader → splitter → faiss_store → (mmr / compression / multi-query / dedup) → chain → llm — pick the strategy that matches the symptom.

1. The rag/ folder, in pipeline order

Ten small, single-purpose files replace what used to be one long notebook cell. Reading them in this order matches how the pipeline actually runs, bottom-up: foundations → data prep → storage → retrieval strategies → generation.

File	Stage	Job
embeddings.py	Foundation	Wraps the embedding model client (e.g. OpenAI embeddings)
llm.py	Foundation	Wraps the chat/completion model client used for generation
loader.py	Data prep	Reads uploaded files (PDFs) into raw documents
splitter.py	Data prep	Chunks documents into retrieval-sized pieces
faiss_store.py	Storage	Builds/queries the FAISS vector index over chunk embeddings
mmr.py	Retrieval strategy	Maximal Marginal Relevance — trades relevance for diversity via λ
compression.py	Retrieval strategy	Contextual compression — keep only the relevant span of each chunk
multi_query_retriever.py	Retrieval strategy	Asks the question multiple reworded ways, unions results
deduplicator.py	Retrieval strategy	Removes overlapping/duplicate chunks before they reach the LLM
chain.py	Generation	Assembles retrieved context + question into the final prompt/answer chain

2. Choosing a retrieval strategy

Symptom	Reach for
Top-k results are all near-duplicates of each other	mmr.py (diversify) or deduplicator.py
Bot misses answers you know are in the document	multi_query_retriever.py
Right chunk retrieved, but LLM answer is padded with irrelevant text	compression.py
Same fact retrieved twice from overlapping chunks	deduplicator.py

3. Where evaluation plugs in

evaluation/ragas_eval.py sits right after chain.py in the request flow: retrieval_service.py generates the answer, then immediately scores it (RAGAS / LLM-judge metrics from Week 6) before returning (answer, answer_texts, evaluation_metric) to the API layer. Evaluation is not a separate offline step here — it runs on every real request.

■ Pitfall — Whole-directory reload bug

A real bug hit while building this: uploading a second document caused ingestion_service.py to reload and re-embed the *entire* input directory, so the reported chunk count included the first file's chunks again. The regression test is exactly this: upload file A, note the chunk count, upload file B, and confirm the count reflects only file B.

✓ **Building blocks stay HTTP-agnostic**

Nothing in `rag/` should know it's being called from a FastAPI route. That's what keeps these files unit-testable in isolation and reusable if the orchestration layer changes.

4. Interview Prep

Q: Why split retrieval into four separate files (`mmr`, `compression`, `multi_query`, `deduplicator`) instead of one retriever function?

A: Each addresses a different failure mode (redundancy, noise, recall, overlap) and can be swapped, tuned, or A/B tested independently — combining them into one function would make it unclear which knob to turn when a specific symptom shows up.

Q: Where does evaluation happen in this pipeline and why there?

A: Inside `retrieval_service.py`, immediately after `chain.py` produces an answer — evaluation runs per-request in production, not just offline, so metrics are available for every real answer served.

Q: What's the actual regression test for a chunk-counting bug like the whole-directory reload issue?

A: Upload one document, record the returned chunk count, upload a second document, and confirm the count reflects only the newly uploaded file — not the sum of everything ingested so far.